

Course ID: SW 302

User Interface (UI) Development

Dr. Mohamed Sami Rakha

Lecture#7

DOM, Intro to JavaScript

Lecture Outline

Dr. Mohamed Sami Rakha

Document Object Model (DOM)

- Access DOM Elements
- Change the Content / Style of Elements

Introduction to JavaScript

Recap: Document Object Model (DOM)

Window Interface

5. Window interface (Not a DOM interface but important to mention)

- window is not a DOM interface, it's the container for the DOM.
- it belongs to the **BOM (Browser Object Model)**.
- **Browser controls & metadata** → window.location, window.history, window.navigator.
- **Global functions** → alert(), setTimeout(), setInterval().

Hierarchy

Window (BOM)

- └ Document (DOM root)
 - └ Elements / Nodes
 - └ Text, Attributes, etc.

// Accessing window properties

```
console.log(window.innerWidth, window.innerHeight); // viewport size
```

// Using window functions

```
window.alert("Hello!");
```

// Access DOM via window

```
console.log(window.document.title);
```

// Timer

```
window.setTimeout(() => {  
  console.log("Executed after 2 seconds");  
}, 2000);
```

Window Interface

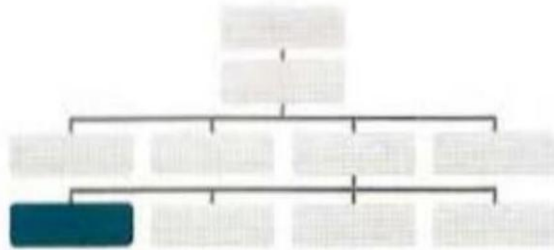
Function / Property	Description
<code>alert(message)</code>	Shows a pop-up alert box.
<code>confirm(message)</code>	Shows an OK/Cancel dialog, returns true/false.
<code>prompt(message)</code>	Shows input dialog, returns user input.
<code>setTimeout(fn, ms)</code>	Executes function once after delay.
<code>setInterval(fn, ms)</code>	Repeats function every interval.
<code>clearTimeout(id)</code>	Cancels a timeout.
<code>clearInterval(id)</code>	Cancels an interval.
<code>location</code>	Provides info about current URL (window.location.href).
<code>history</code>	Allows navigation (back/forward).
<code>localStorage / sessionStorage</code>	Store key/value pairs in the browser.

Access DOM Elements

Access DOM Elements

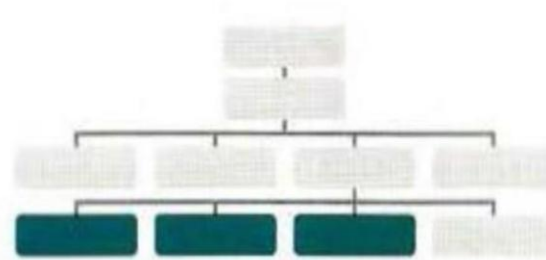
The document is the root node of the DOM tree and offers many useful properties and methods. **document.getElementById(str)** gives you the element with str as id (or name). It returns a reference to the DOM tree node representing the desired element.

1- SELECT AN INDIVIDUAL ELEMENT NODE



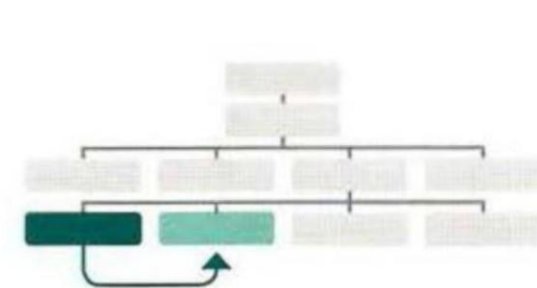
getElementById()
querySelector()

2- SELECT MULTIPLE ELEMENTS (NODELISTS)



getElementsByClassName()
getElementsByTagName()
querySelectorAll()

3- TRAVERSING BETWEEN ELEMENT NODES



parentNode
previousSibling/nextSibling
firstChild / lastChild

Access a Single Element Node

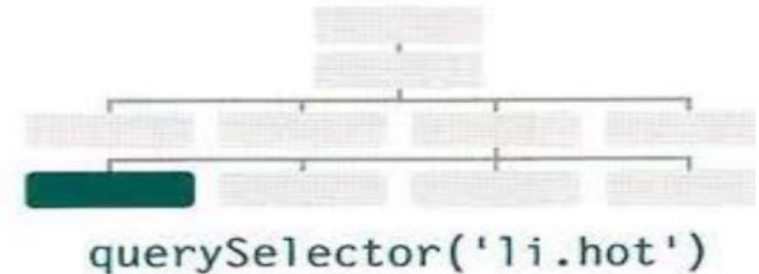
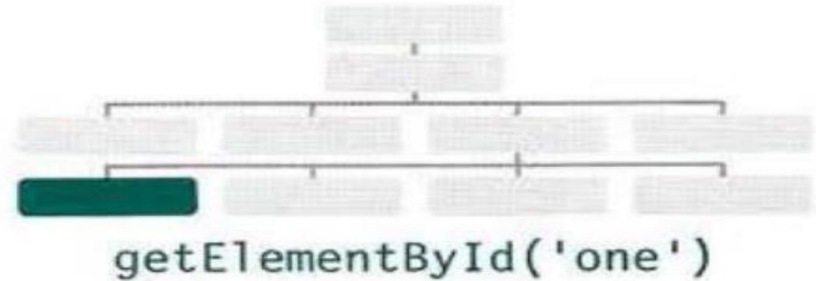
1. `getElementById('id')`:

Selects an individual element given the value of its id attribute. The HTML element must have an id attribute in order to be selectable.

2. `querySelector('css selector')`:

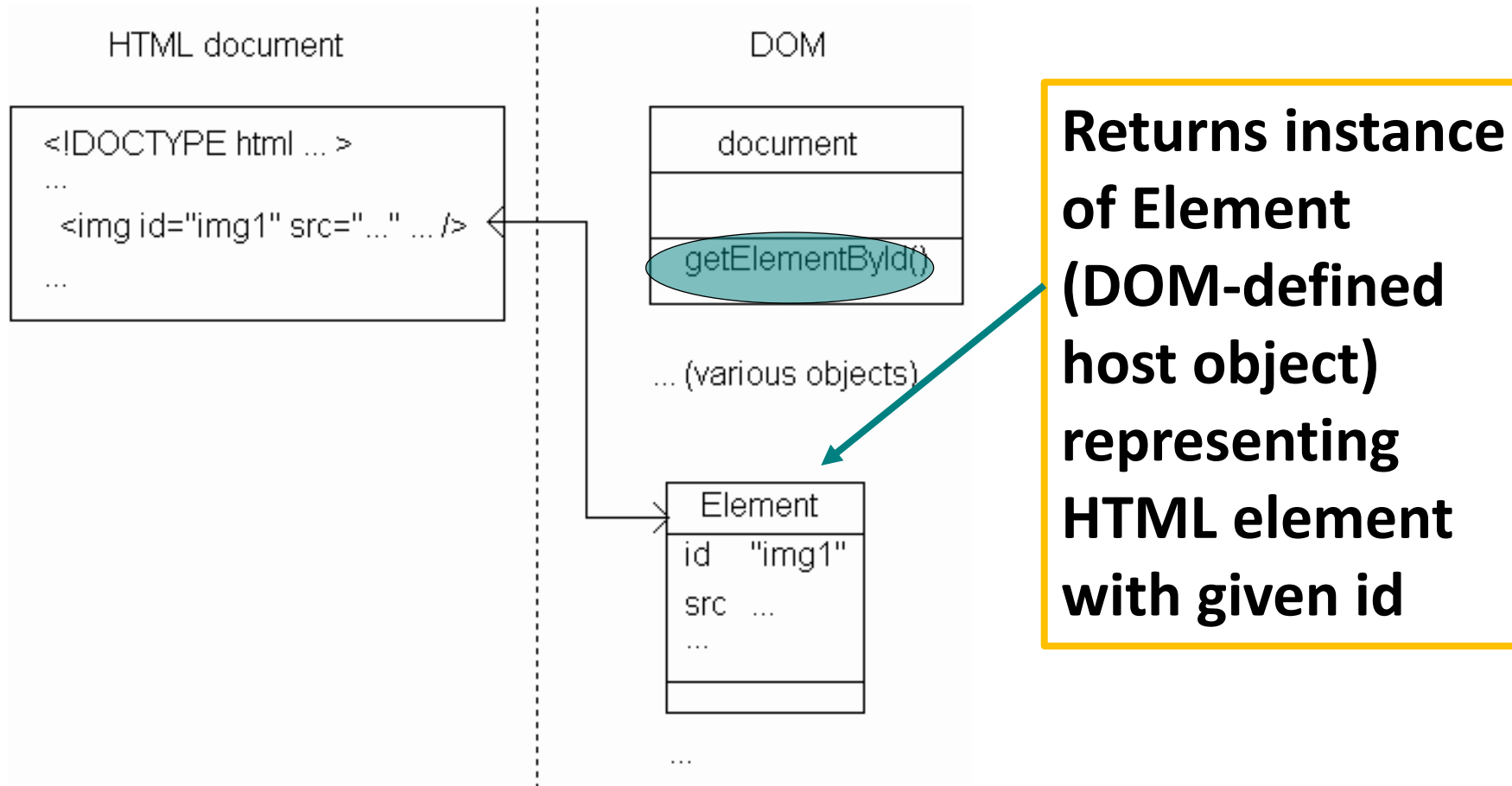
Uses CSS selector syntax that would select one or more elements. This method returns **only the first matching element**.

Dr. Mohamed Sami Rakha



Access a Single Element Node

document.getElementById(str) gives you the element with str as id (or name). It returns a reference to the **DOM tree node** representing the desired element.



Document vs Element Interfaces

Document interface is the global entry point. You use it to find or create elements. Once you have an element, you use the **Element interface** to **change its content, attributes, or style**.

Feature	Document Interface	Element Interface
Definition	Represents the entire HTML document.	Represents a single HTML element (like <p>, <div>, <h1>).
Scope	The root entry point into the DOM tree.	A node inside the DOM tree.
Usage	Used to find, create, or access elements.	Used to manipulate properties & attributes of a specific element.
Examples	<pre>js
document.getElementById("title")
>document.querySelector("p")
document.createElement("div")
</pre>	<pre>js
element.innerHTML = "Hello"
element.setAttribute("class","box")
>element.style.color = "red"
</pre>
Analogy	Like a library catalog → you search for a book.	Like a book itself → you can read or edit it.

Access and Update Example

```
<body>
  <h1 id="title">Hello World</h1>
  <p id="para">This is a paragraph.</p>

  <script>
    // Using the `document` interface (global entry point)
    var heading = document.getElementById("title"); // Finds <h1>
    var paragraph = document.querySelector("#para"); // Finds <p>

    // Now we have ELEMENTS, we can use the Element interface on them
    heading.textContent = "Updated Heading"; // Modify text
    paragraph.style.color = "blue";          // Change CSS style
    paragraph.setAttribute("class", "highlight"); // Add an attribute
  </script>
</body>
```

Once you have “heading” or “paragraph” element, it’s an **Element object**. You can now use the Element interface to:

- **Change its text (.textContent).**
- **Change style (.style.color).**
- **Add attributes (.setAttribute).**

Access One or More Elements

1. `getElementsByClassName('class')`

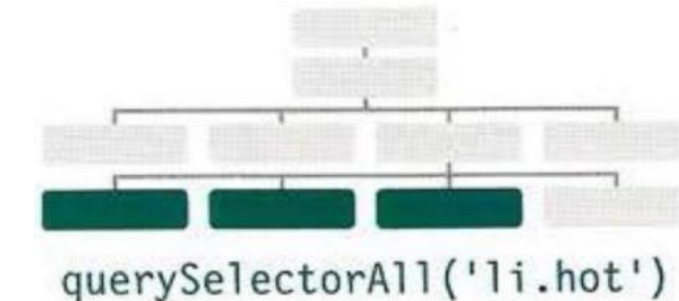
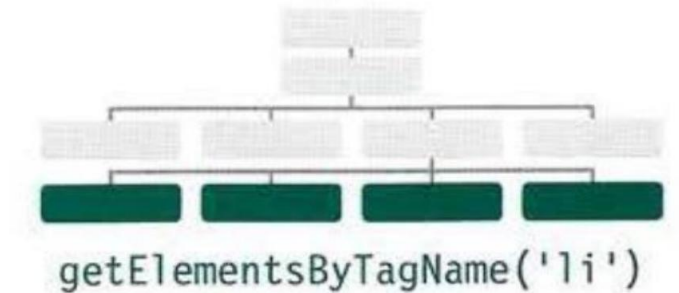
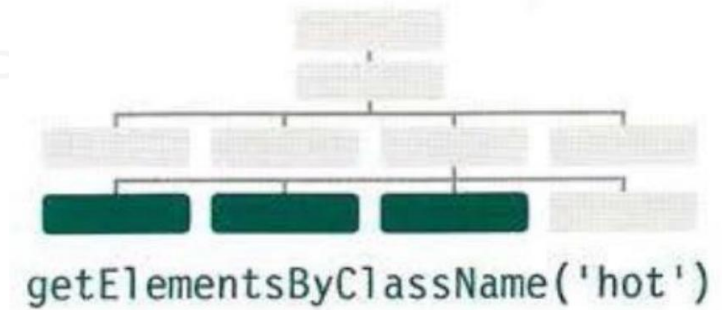
Selects one or more elements given the value of their class attribute. The HTML must have a class attribute for it to be selectable. This method is faster than `querySelectorAll()`.

2. `getElementsByTagName('tagName')`

Selects all elements on the page with the specified tag name. This method is faster than `querySelectorAll()`.

3. `querySelectorAll('css selector')`

Uses CSS selector syntax to select one or more elements and returns all of those that match.



Access One or More Elements

1. `getElementsByClassName('class')`

- Selects all elements with the specified class.
- Returns a live HTMLCollection (updates if DOM changes).

2. `getElementsByTagName('tagName')`

- Selects all elements with the specified tag (e.g., div, p).
- Returns a live HTMLCollection.

3. `querySelectorAll('css selector')`

- Selects elements using CSS selectors (**id, class, tag, attribute, etc.**).
- Returns a static NodeList (does not update when DOM changes).
- **More flexible but slower.**

```
<p class="note">Hello</p>
<p class="note">World</p>
<script>
  let notes = document.getElementsByClassName("note");
  console.log(notes.length); // print 2
</script>
```

```
<p>Paragraph 1</p>
<p>Paragraph 2</p>
<script>
  let paras = document.getElementsByTagName("p");
  console.log(paras[0].innerText); // "Paragraph 1"
</script>
```

```
<div class="box"></div>
<div class="box"></div>
<script>
  let boxes = document.querySelectorAll(".box");
  console.log(boxes.length); // 2
</script>
```

Change the Content of Elements

Change the Content of Elements

JavaScript can dynamically **change the text or HTML** of elements in the DOM

1- element.innerHTML

- Sets or gets the HTML content of an element.
- Can insert HTML tags (be cautious about security).

```
document.getElementById("demo").innerHTML = "<b>Hello  
World</b>";
```

2- element.textContent

Sets or gets the plain text (ignores HTML tags).

```
document.getElementById("demo").textContent = "Hello  
World";
```

3- element.innerText

Similar to textContent but respects CSS (hidden text not returned).

```
document.getElementById("demo").innerText = "Hello  
Students!";
```

When to use element.innerText?

What does it do again? Gets or sets the *visible* text inside an element, taking into account **CSS styling, hidden elements, and line breaks**.

- If you only care about the **rendered text** (the text that the user *sees*).
- If you want JavaScript to **respect things like** `display: none`, `visibility: hidden`, or CSS formatting (like line breaks).
- Good for **user-facing text updates** where visual appearance matters.

```
<p id="demo">
  Hello <span style="display:none">Secret</span> World!
</p>

<script>
  const el = document.getElementById("demo");
  console.log(el.textContent); // "Hello Secret World!"
  console.log(el.innerText);  // "Hello World!" (hidden span
                              ignored)
</script>
```

Please remember :

innerText → what the user sees (rendered text, obeys CSS).

textContent → raw text in the DOM (ignores CSS, includes hidden content).

innerHTML → actual HTML markup (tags + text).

JavaScript

DOM Connects all (HTML, CSS, JavaScript)



DOM The interface that connects
HTML (structure), CSS (style),
and JavaScript (behavior)

What JavaScript

- **JavaScript** is a client-side scripting language; it is processed on the user's machine (not the server).
- It is reliant on the browser's capabilities (it may even be unavailable entirely).
- It is a **dynamic programming language**, it does not need to be compiled into an executable program. The browser reads it just as we do.
- It is **loosely typed**, you don't need to define variable types as you do for other programming languages.

JavaScript Tasks

JavaScript adds a behavioral layer (interactivity) to a web page. Some examples include:

- Checking form submissions and providing feedback messages and UI changes
- Injecting content into current documents on the fly
- Showing and hiding content based on a user clicking a link or heading
- Completing a term in a search box
- Testing for browser features and capabilities
- Much more!

Adding Scripts to a Page

Embedded script

Dr. Mohamed Sami Rakha

Include the script in an HTML document with the script element:

```
<script>  
    ... JavaScript code goes here  
</script>
```

External script

Use the src attribute in the script element to point to an external, standalone .js file:

```
<script src="my_script.js"></script>
```

Script Placement

Script Placement:

Dr. Mohamed Sami Rakha

The **script** element can go anywhere in the document, but the most common places are as follows:

In the head of the document

For when you want the script to do something before the body completely loads (ex: Modernizr):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <script src="script.js"></script>
</head>
...
```

Just before the </body> tag

Preferred when the browser needs to parse the document and its DOM structure before running the script:

```
...
<body>
  <!-- contents of page -->
  <script src="script.js"></script>
</body>
</html>
```

JavaScript Syntax Basics

JavaScript is case sensitive

Dr. Mohamed Sami Rakha

White space is ignored (unless it is enclosed in quotes in a text string).

A script is made up of a series of statements, commands that tell the browser what to do.

Semicolons are Optional if your statements are each placed on a separate line

Single-line comments in JavaScript appear after two `//` characters:

```
// This is a single-line comment
```

Multiple-line comments go between `/*` and `*/` characters.

Building Blocks of Scripts

Dr. Mohamed Sami Rakha

- Variables
- Comparison operators
- if/else statements
- Loops
- Functions
- Scope

Variables

A **variable** is made up of a **name** and a **value**.

Dr. Mohamed Sami Rakha

You create a variable so that you can refer to the value by name later in the script.

- The value can be a number, text string, element in the DOM, or function, to name a few examples.
- Variables are defined using the **var** keyword:

```
var foo = 5;
```

The variable is named `foo`. The equals sign (=) indicates we are **assigning** it the numeric value of 5.

Variables

Rules for naming a variable:

Dr. Mohamed Sami Rakha

- It must start with a letter or underscore.
- It may not contain character spaces. Use underscores or CamelCase instead.
- It may not contain special characters (! . , / \+ * =).
- It should describe the information it contains.

Value Data Types

Values assigned to variables fall under a few **data types**:

Undefined

The variable is declared by giving it a name, but no value:

```
var foo;  
  
alert(foo); // Will open a dialog containing "undefined"
```

null

Assigns the variable no inherent value:

```
var foo = null;  
  
alert(foo); // Will open a dialog containing "null"
```

Numbers

When you assign a number (e.g., 5), JavaScript treats it as a number (you don't need to tell it it's a number):

```
var foo = 5;  
  
alert(foo + foo); // This will alert "10"
```

Value Data Types

Strings

If the value is wrapped in single or double quotes, it is treated as a string of text:

```
var foo = "five";  
alert(foo); // Will alert "five"  
alert(foo + foo); // Will alert "fivefive"
```

Booleans

Assigns a true or false value, used for scripting logic:

```
var foo = true; // The variable "foo" is now true
```

Arrays

A group of multiple values (called *members*) assigned to a single variable. Values in arrays are *indexed* (assigned a number starting with 0). You can refer to array values by their index numbers:

```
var foo = [5, "five", "5"];  
  
alert( foo[0] ); // Alerts "5"  
alert( foo[1] ); // Alerts "five"  
alert( foo[2] ); // Also alerts "5"
```

Array Methods

Usage of Array Methods: “Push” and “Pop”

Dr. Mohamed Sami Rakha

// push(): Adds an element to the end of an array.

```
colors.push("purple");
```

```
console.log(colors);
```

// Output: ["red", "blue", "yellow", "purple"]

// pop(): Removes the last element from an array.

```
colors.pop();
```

```
console.log(colors);
```

// Output: ["red", "blue", "yellow"]

Array Methods

Usage of Array Methods: “Shift” and “Unshift”

Mohamed Sami Rakha

```
// shift(): Removes the first element from an array.
```

```
colors.shift();
```

```
console.log(colors);
```

```
// Output: ["blue", "yellow"]
```

```
// unshift(): Adds an element to the beginning of an array.
```

```
colors.unshift("pink");
```

```
console.log(colors);
```

```
// Output: ["pink", "blue", "yellow"]
```

Array Methods

Usage of Array Methods: “indexOf()” and “includes()”

// indexOf(): Returns the first index at which a given element can be found.

```
console.log(colors.indexOf("blue")); // Output: 1
```

```
console.log(colors.indexOf("pink")); // Output: 0
```

// includes(): Checks if an array contains a certain element (returns true or false).

```
console.log(colors.includes("yellow")); // Output: true
```

```
console.log(colors.includes("black")); // Output: false
```

Array Methods

Usage of Array Methods: “**sort()**” and “**reverse()**”

ed Sami Rakha

// sort(): Sorts the elements of an array in place and returns the sorted array.

```
var numbers = [3, 1, 4, 1, 5];
```

```
numbers.sort();
```

```
console.log(numbers);
```

// Output: [1, 1, 3, 4, 5]

// reverse(): Reverses the order of the elements in an array in place.

```
numbers.reverse();
```

```
console.log(numbers);
```

// Output: [5, 4, 3, 1, 1]

Array Methods

Usage of Array Methods: “**join()**” and “**toString()**”

// join(): Joins all elements of an array into a string, separated by a specified separator.

```
var colors = ["orange", "green", "yellow"];
```

```
var colorString = colors.join(" ");
```

```
console.log(colorString);
```

// Output: orange green yellow

// toString(): Returns a string representing the specified array and its elements.

```
console.log(colors.toString());
```

// Output: orange,green,yellow

Array Methods

Usage of Array Methods: “**slice()**” : Creates a new array by extracting a section of the original array (does not modify the original).

```
var colors = ["orange", "green", "yellow"];  
var newColors = colors.slice(0, 2); // Extracts elements from index 0 to 1  
console.log(newColors);  
// Output: ["orange", "green"]
```

Value Data Types

Introducing the **Object Type** in JavaScript

```
var student = {  
  name: "John",  
  age: 20,  
  courses: ["Math", "Science"]  
};  
// Accessing object properties  
console.log(student.name); // Output: John
```

Defines an object with multiple property types: string (name), number (age), and array (courses).

Shows how to access a specific property using dot notation (student.name).

Adding or modifying a property:

```
student.grade = "A"; // Adding a new property  
student.age = 21; // Modifying an existing property
```

Accessing using bracket notation (useful for dynamic keys):

```
console.log(student["name"]); // Output: John
```

Comparison Operators

Comparison operators are special characters in JavaScript syntax that evaluate and compare values:

<code>==</code>	Is equal to
<code>!=</code>	Is not equal to
<code>===</code>	Is identical to (equal to and of the same data type)
<code>!==</code>	Is not identical to
<code>></code>	Is greater than
<code>>=</code>	Is greater than or equal to
<code><</code>	Is less than
<code><=</code>	Is less than or equal to

Comparison Operators

Example

When we compare two values, JavaScript evaluates the statement and gives back a Boolean (true/false) value:

```
alert( 5 == 5 ); // This will alert "true"
alert( 5 != 6 ); // This will alert "true"
alert( 5 < 1 );  // This will alert "false"
```

NOTE: Equal to (==) is not the same as identical to (===). Identical values must share a data type:

```
alert( "5" == 5 ); // This will alert "true". They're both "5".
```

```
alert( "5" === 5 ); // This will alert "false". They're both
"5", but they're not the same data type.
```

```
alert( "5" !== 5 ); // This will alert "true", since they're
not the same data type.
```

if/else Statements

An if/else statement

tests for conditions by asking a true/false question.

If the condition in parentheses is met, then execute the commands between the curly brackets ({}):

Dr. Mohamed Sami Rakha

```
if(true) {  
    // Do something.  
}
```

Example:

```
if( 1 != 2 ) {  
    alert("These values are not equal.");  
    // It is true that 1 is never equal to 2, so we should see  
    this alert.  
}
```

if/else Statements

If you want to do one thing if the test is true and something else if it is false, include an **else statement** after the if statement:

```
var test = "testing";  
if( test == "testing" ) {  
    alert( "You haven't changed anything." );  
} else {  
    alert( "You've changed something!" );  
}
```

Changing the value of the test variable to anything but the word “testing” will trigger the alert “You've changed something!”

Conditional Statements

uses a **switch** statement to compare the value of the variable `day` against multiple case options. When `day` matches 3, it prints "Wednesday" to the console. If no cases match, the default block runs.

```
var day = 3;

switch (day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday");
    break;
  default:
    console.log("Not a valid day");
}
```

Loops

Loops allow you to do something to every variable in an array without writing a statement for every one.

One way to write a loop is with a **for statement**:

```
for(initialize variable; test condition; alter the value;) {  
    // do something  
}
```

Loops

Example: This loop triggers **3 alerts**, reading "0", "1", and "2":

```
for(var i = 0, i <= 2, i++) {  
    alert(i);  
}
```

- `for()`: Says, "for every time this is true, do this."
- `var i = 0`: Creates a new variable `i` with its value set to 0. "i" (short for "index") is a common variable name.
- `i <= 2`: Says, "as long as `i` is less than or equal to 2, keep looping."
- `i++`: Shorthand for "every time this loop runs, add 1 to the value of `i`."
- `{alert(i);}`: This loop will run three times (once each for 0, 1, and 2 values) and alert the `i` value.

Loops

This code demonstrates a **while loop** in JavaScript. The loop continues to run as long as the condition $i < 5$ is true. On each iteration, it prints the current value of i and then increments i by 1.

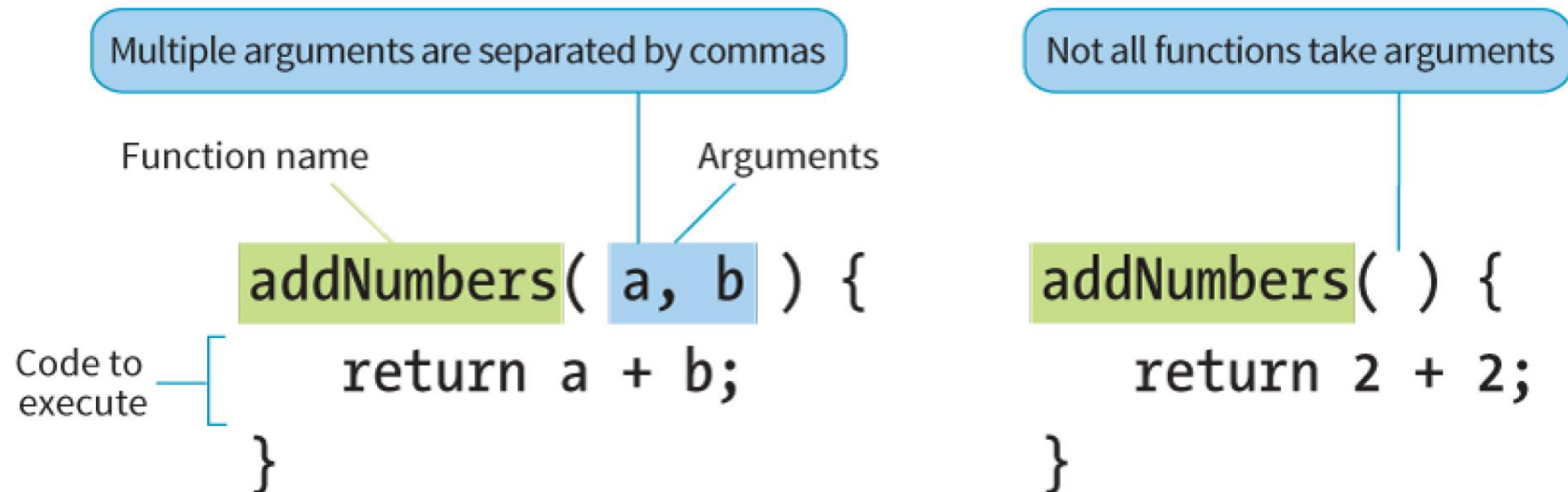
```
var i = 0;

while (i < 5) {
    console.log("Iteration " + i);
    i++;
}
```

Functions

A function is a bit of code for performing a task that doesn't run until it is referenced or called.

Parentheses sometimes contain **arguments** (additional information used by the function):



Functions (cont-d)

Some functions are built into JavaScript. Here are examples of **native functions**:

- `alert()`, `confirm()`, and `prompt()`
Functions that trigger browser-level dialog boxes
- `Date()`
Returns the current date and time

You can also create your own **custom functions** by typing **function** followed by a name for the function and the task it performs:

```
function name() {  
    // Code for the new function goes here.  
}
```

Arrow Functions

Arrow functions are a shorter and more modern way to write functions in JavaScript.... are especially useful when writing short, simple functions.

//for single line
code {} are not
necessary

Multiline Arrow
Function with
var

```
var add = (a, b) => a + b;  
console.log(add(3, 4)); // Output: 7
```

```
var getUserInfo = (name, age) => {  
  var greeting = `Hello, my name is ${name}.`;   
  var info = `I am ${age} years old.`;  
  return `${greeting} ${info}`;  
};  
  
console.log(getUserInfo("John", 25));  
// Output: Hello, my name is John. I am 25 years old.
```

Variable Scope

A variable that can only be used within one function is **locally scoped (function-scoped)**. When you define a variable inside a function, include the **var** keyword to keep it locally scoped (recommended):

```
var foo = "value";
```

A variable that can be used by any script on your page is said to be **globally scoped**.

- Any variable created *outside* of a function is automatically globally scoped:

```
var foo = "value";
```

- To make a variable created *inside* a function globally scoped, omit the **var** keyword:

```
foo = "value";
```

What is let in JavaScript?

let is used to declare block-scoped variables (introduced in ES6). It's an improvement over **var**, which is function-scoped and can behave unpredictably in certain cases due to **hoisting**.

Key Differences from **var**:

Block Scope: **let** is only available within the { } in which it is defined. Not “**Locally Scoped**”

No Redeclaration: You can't redeclare a **let** variable in the same scope.

```
if (true) {  
  let x = 10;  
  console.log(x); // 10  
}  
console.log(x); // ❌ Error: x is not defined
```

```
let name = "John";  
let name = "Jake"; // ❌ Error
```

Locally Scoped vs Block Scope

var is Locally Scoped (Function-Scoped)

```
function testVar() {  
  if (true) {  
    var x = 10;  
  }  
  console.log(x); //  10 (x is visible outside the block)  
}  
testVar();
```

var x is accessible anywhere inside the function, even outside the if.

let is Block-Scoped

```
function testLet() {  
  if (true) {  
    let x = 10;  
  }  
  console.log(x); //  Error: x is not defined  
}  
testLet();
```

let x only exists inside the { } block.

What is Hoisting in JavaScript?

Hoisting is a JavaScript mechanism where variable and function declarations **are moved** to the **top of their scope during the compilation phase**, before the code is executed.



You can use functions and variables *before they are declared* in the code, because JavaScript “**hoists**” their declarations.

Hoisting with Function Declarations:

```
greet(); // ✅ Works because the function is hoisted
function greet() {
  console.log("Hello!");
}
```

Hoisting with var:

```
console.log(x); // Output: undefined
var x = 10;
```

- Only the declaration of x is hoisted (var x), not its initialization (= 10).
- That's why x exists but holds the value undefined.

initialized to undefined, so no error.

var vs **let**: During Hoisting

```
console.log(a); // Output: undefined
var a = 5;

console.log(b); // ✗ Error: Cannot access 'b' before initialization
let b = 5;
```

var a is hoisted and initialized to undefined, so no error.

let b is hoisted but not initialized, so trying to access it before the declaration causes an error.

- Both let and var are hoisted.
- Only var is initialized to undefined during hoisting, making it accessible before the declaration (but potentially buggy).
- **let is hoisted but not initialized, so using it before declaration throws an error.**

Define Constant

In JavaScript, you can define a constant using the **const** keyword. A constant is a variable whose value cannot be reassigned once it's set. Here's the basic syntax:

```
const PI = 3.14159;
```

You must assign a value when you declare a **const**. This will throw an error:

```
const x; // ❌ Error: Missing initializer
```

You cannot reassign a constant:

```
const name = "Mo";  
name = "Rakha"; // ❌ Error: Assignment to constant variable
```

Const is hoisted with the same rules as "let"

Define Constant

The value itself is not always **immutable** (unalterable). For example, objects or arrays declared with `const` can still have their contents changed:

```
const person = { name: "Mo" };  
person.name = "Rakha"; //  This is allowed
```

Use **const** when you don't intend to reassign the variable—it's a best practice for safer, more predictable code.

Dr. Mohamed Sami Rakha

Thank You...